S06 Menus

CRE132 worksheet

What you'll make

The menus every game needs: a **main menu** with Play, Settings and Quit, a **pause menu** that freezes the game, and a **Game Over** screen when the player dies. Every menu works with the mouse, the arrow keys and Enter, or a gamepad.

By the end you'll know how menus are just panels switched on and off, how `Time.timeScale` pauses a game, and how to make a menu work without a mouse.

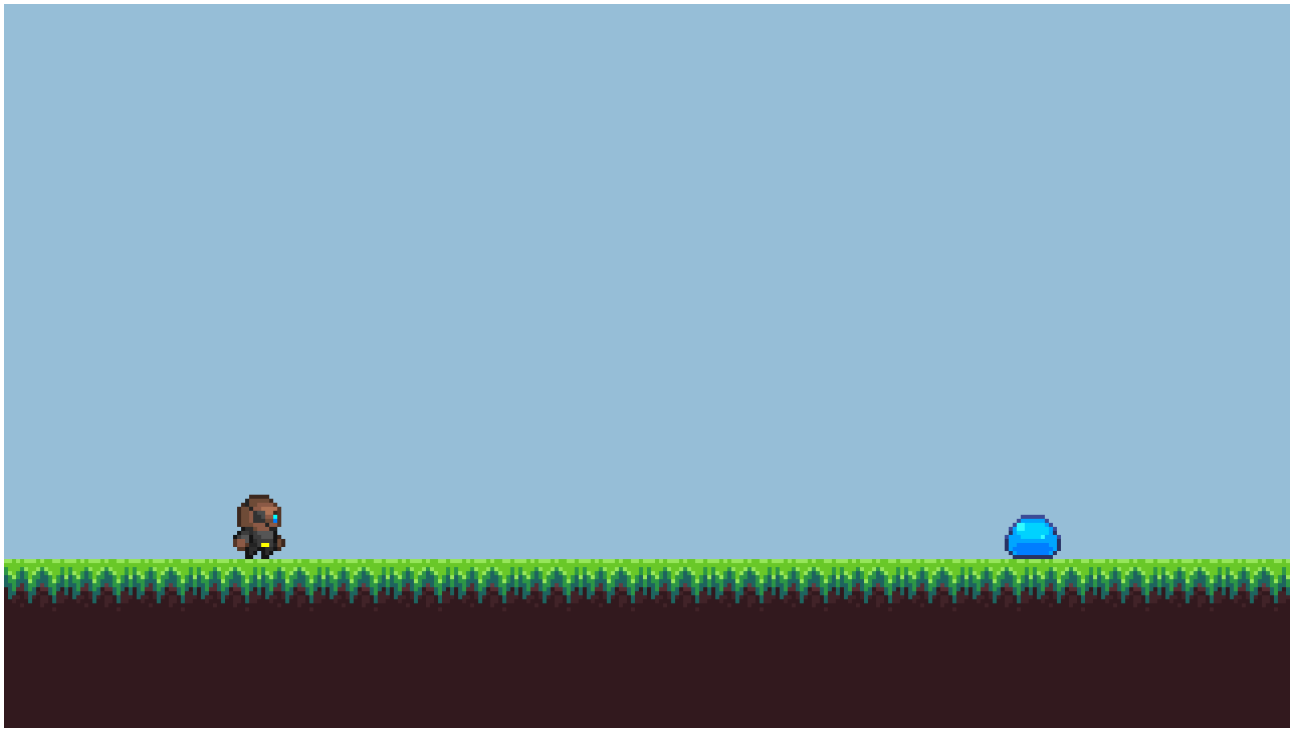


Figure 1: The finished menu scene's background: the hero and a slime in a meadow.

Before you start

1. Unzip `CRE132_S06_Menus_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S06_Menus_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S06_Menus` (the main menu). The game is `S06_Menus_Game` in the same folder; both are in the build list.
3. Press **Play**. The music starts and you see **Slime Meadow** with **Play**, **Settings** and **Quit** buttons, but they do nothing. Stop, open `S06_Menus_Game` and press Play: it's S05's room, but Escape does nothing and dying just leaves an empty level.

In both scenes, the menus are already laid out on a **Menu Canvas**: in the main menu a **Main Panel** and a hidden **Settings Panel**; in the game a **Pause Panel**, a **Settings Panel** and a **Game Over Panel**, all hidden. The buttons' **On Click ()** lists are empty: you connect each one to its method. Each scene also has a GameManager, an AudioManager and a Scene Loader.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Three small additions

The menus need a few things the managers can't do yet. All three scripts are in `Assets/_game/Scripts`.

GameManager (`Managers` folder): a new game should start at 0. Add this method at the end of the class, below `AddScore` (before the last `}`):

```
    // Back to zero for a new game (the main menu's Play button calls this).
    public void ResetScore()
    {
        score = 0;
    }
```

AudioManager (`Managers` folder): the Settings sliders need to know the current volume. Add this method below `SetVolume`, at the end of the class:

```
    // The other way: decibels from the mixer back to a 0 to 1 slider value.
    public float GetVolume(string exposedParameter)
    {
        float decibels;
        mixer.GetFloat(exposedParameter, out decibels);
        return Mathf.Pow(10f, decibels / 20f);
    }
```

VolumeSlider (`UI` folder): in S05 every slider started at full and set the volume to full when a scene loaded. Now it should start wherever the mixer is. Replace the whole `Start` method with:

```
    private void Start()
    {
        slider = GetComponent<Slider>();

        // Start where the mixer is now, so loading a scene doesn't reset the volume
        // to full.
        slider.value = AudioManager.Instance.GetVolume(exposedParameter);

        // Call SetVolume whenever the slider moves. This is the same list as On Value
        // Changed in the Inspector.
        slider.onValueChanged.AddListener(SetVolume);
    }
```

- **`mixer.GetFloat(..., out decibels)`** reads a value back from the mixer. `out` means the method fills in the variable you give it.
- **`Mathf.Pow(10f, decibels / 20f)`** undoes S05's `Log10(value) * 20`, turning decibels back into 0 to 1.
- **The old `SetVolume(slider.value);` line has gone**, because the slider now starts at the mixer's value, so there's nothing to set.

2. The main menu: MainMenu

Create `MainMenu` in `Assets/_game/Scripts/UI`:

```
// The main menu: Play, Settings and Quit buttons call these public methods (set in
// each Button's On Click).
```

```
using UnityEngine;
using UnityEngine.EventSystems;

public class MainMenu : MonoBehaviour
{
    [SerializeField] private string gameScene = "S06_Menus_Game";
    [SerializeField] private GameObject mainPanel;
    [SerializeField] private GameObject settingsPanel;

    // The button to highlight when each panel opens, so the menu also works with a
    // gamepad or the arrow keys.
    [SerializeField] private GameObject firstMainButton;
    [SerializeField] private GameObject firstSettingsButton;

    private void Start()
    {
        CloseSettings();
    }

    public void PlayGame()
    {
        GameManager.Instance.ResetScore();
        SceneLoader.Instance.LoadScene(gameScene);
    }

    public void OpenSettings()
    {
        mainPanel.SetActive(false);
        settingsPanel.SetActive(true);
        EventSystem.current.SetSelectedGameObject(firstSettingsButton);
    }

    public void CloseSettings()
    {
        settingsPanel.SetActive(false);
        mainPanel.SetActive(true);
        EventSystem.current.SetSelectedGameObject(firstMainButton);
    }

    public void QuitGame()
    {
        // Quit only works in a built game. In the editor, this log shows the button
        // worked.
        Debug.Log("Quit");
        Application.Quit();
    }
}
```

What's going on:

- **Menus are panels.** `SetActive(false)` hides a panel and everything on it; `SetActive(true)` shows it. Settings hides the main panel and shows the settings panel; Back does the reverse.
- **Every method is public**, so the buttons can call them from **On Click ()** with no code of their own.
- **`PlayGame`** resets the score, then asks S04's Scene Loader for the game scene. The managers from the menu scene come along (DontDestroyOnLoad), and the copies in the game scene destroy themselves.
- **`SetSelectedGameObject`** highlights a button when a panel opens. Without it, keyboard and gamepad players have nothing selected and can't use the menu.

Add it: in `S06_Menus`, add **Main Menu** to **Menu Canvas**. Leave **Game Scene** as `S06_Menus_Game`. Drag **Main Panel** and **Settings Panel** into their slots, **Play Button** into **First Main Button** and **Back Button** (inside Settings Panel) into **First Settings Button**.

Then wire the buttons. For each one, click **+** in its **On Click ()** list, drag **Menu Canvas** into the new entry's object slot (**None**) and pick the method from the dropdown:

Button	Method
Play Button	MainMenu > PlayGame
Settings Button	MainMenu > OpenSettings
Quit Button	MainMenu > QuitGame
Back Button	MainMenu > CloseSettings

Test it: press Play. Settings shows the sliders and Back returns. Quit prints `Quit` in the Console. Use the arrow keys and Enter instead of the mouse: the selected button is red. Play fades into the game, and the music carries on.

3. Pausing: PauseMenu

Create `PauseMenu` in the `UI` folder:

```
// Pause with Escape, P or the gamepad Start button. Pausing sets Time.timeScale to 0,
// which stops physics, animation and anything that uses Time.deltaTime.
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.InputSystem;

public class PauseMenu : MonoBehaviour
{
    [SerializeField] private InputActionReference pauseAction;
    [SerializeField] private string mainMenuScene = "S06_Menus";
    [SerializeField] private GameObject pausePanel;
    [SerializeField] private GameObject settingsPanel;
    [SerializeField] private GameObject firstPauseButton;
    [SerializeField] private GameObject firstSettingsButton;

    private bool isPaused;

    private void OnEnable()
    {
        pauseAction.action.Enable();
    }

    private void OnDisable()
    {
        pauseAction.action.Disable();
    }

    private void Start()
    {
        pausePanel.SetActive(false);
        settingsPanel.SetActive(false);
    }

    private void Update()
    {
```

```
        if (pauseAction.action.WasPressedThisFrame())
        {
            if (isPaused)
            {
                Resume();
            }
            else
            {
                Pause();
            }
        }
    }

    public void Pause()
    {
        isPaused = true;
        Time.timeScale = 0f;
        pausePanel.SetActive(true);
        EventSystem.current.SetSelectedGameObject(firstPauseButton);
    }

    public void Resume()
    {
        isPaused = false;
        Time.timeScale = 1f;
        pausePanel.SetActive(false);
        settingsPanel.SetActive(false);
    }

    public void OpenSettings()
    {
        pausePanel.SetActive(false);
        settingsPanel.SetActive(true);
        EventSystem.current.SetSelectedGameObject(firstSettingsButton);
    }

    public void CloseSettings()
    {
        settingsPanel.SetActive(false);
        pausePanel.SetActive(true);
        EventSystem.current.SetSelectedGameObject(firstPauseButton);
    }

    public void GoToMainMenu()
    {
        // timeScale is not reset by loading a scene, so set it back first or the menu
        // would be frozen too.
        Time.timeScale = 1f;
        SceneLoader.Instance.LoadScene(mainMenuScene);
    }
}
```

- **`Time.timeScale = 0`** freezes physics, animation, particles and anything that moves with `Time.deltaTime`. `Update` still runs, and so does input, which is how the pause key can unpause.
- **`timeScale` survives scene loads.** `GoToMainMenu` sets it back to 1 first, or the main menu would be frozen too. (S04's fade uses `Time.unscaledDeltaTime`, so it works either way.)
- **The Pause Panel's dim background** covers the whole screen, so it also blocks clicks on the HUD's Restart button.

First, a Pause action. Double-click `CRE132Controls` (in `Assets/_game/Input`) and add an action called `Pause` (**Button**) with three bindings: **Escape [Keyboard]**, **P [Keyboard]** and **Start [Gamepad]**. Set the first binding's **Path** with **Listen**, then use **+ > Add Binding** for the other two. Click **Save Asset** at the top of the window (or tick **Auto-Save**) and close the editor.

Add it: open `S06_Menus_Game` and add **Pause Menu** to **Menu Canvas**. Set **Pause Action** to **Player/Pause** (Escape, P or the gamepad's Start button). Leave **Main Menu Scene** as `S06_Menus`. Drag in **Pause Panel**, **Settings Panel**, **Resume Button** (**First Pause Button**) and **Back Button** (**First Settings Button**). Wire the buttons with **Menu Canvas** as before:

Button	Method
Resume Button	PauseMenu > Resume
Settings Button	PauseMenu > OpenSettings
Main Menu Button (in Pause Panel)	PauseMenu > GoToMainMenu
Back Button	PauseMenu > CloseSettings

Test it: press Play and Escape. The fire, the animations and the player freeze. Press Escape again, or click Resume, to carry on. Change the volume in Settings, go to the main menu and back: the sliders remember it.

4. Game over: GameOverScreen

Create `GameOverScreen` in the `UI` folder:

```
// Shows the Game Over panel when the player has died (its Health has been destroyed).
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;

public class GameOverScreen : MonoBehaviour
{
    [SerializeField] private Health playerHealth;
    [SerializeField] private string mainMenuScene = "S06_Menus";
    [SerializeField] private GameObject gameOverPanel;
    [SerializeField] private GameObject firstButton;

    private bool isShowing;

    private void Start()
    {
        gameOverPanel.SetActive(false);
    }

    private void Update()
    {
        // A destroyed object compares equal to null in Unity, so this is true once
        // the player has died.
        if (!isShowing && playerHealth == null)
        {
            isShowing = true;
            gameOverPanel.SetActive(true);
            EventSystem.current.SetSelectedGameObject(firstButton);
        }
    }

    public void Restart()
```

```
    {
        GameManager.Instance.ResetScore();
        SceneLoader.Instance.LoadScene(SceneManager.GetActiveScene().name);
    }

    public void GoToMainMenu()
    {
        SceneLoader.Instance.LoadScene(mainMenuScene);
    }
}
```

- **`playerHealth == null`:** S01's Health destroys the player at zero, and Unity treats a destroyed object as `null`. That's how the screen knows the player has died.
- **`isShowing`** makes sure the panel only opens once, instead of every frame.

Add it to **Menu Canvas** too. Drag the **Player** into **Player Health**, **Game Over Panel** into its slot and **Try Again Button** into **First Button**. Wire **Try Again Button** to **GameOverScreen > Restart** and the **Main Menu Button** in Game Over Panel to **GameOverScreen > GoToMainMenu**.

Test it: stand in the fire until you die. Game Over appears with Try Again selected. Try Again restarts the level with the score at
0.

Check your work

- ☐ Start from `S06_Menus`: Play, Settings, Back and Quit all work, with the mouse and with the arrow keys and Enter.
- ☐ `CRE132Controls` has a **Pause** action bound to Escape, P and Start.
- ☐ Escape or P pauses and unpauses; Main Menu from the pause menu isn't frozen.
- ☐ Dying shows Game Over; Try Again and Main Menu both work.
- ☐ The music never restarts, and the volume you set carries between scenes.
- ☐ Compare with `CRE132_S06_Menus_Finished` if something behaves differently.

Key points

- **A menu is a panel** switched on and off with `SetActive`. Buttons call public methods through **On Click ()**.
- **`Time.timeScale`** affects `Time.deltaTime`, FixedUpdate, Animators and particles, but not `Update` itself, input or `Time.unscaledDeltaTime`. Always set it back to 1 before leaving the scene.
- **Always give a menu a first selected button**, or gamepad players can't use it.
- **Menu panels go on their own canvas** with a higher **Sort Order** than the HUD (10 here), so they draw on top.

If something goes wrong

- **A button does nothing, or says "No Function":** drag **Menu Canvas** into its On Click () slot and pick the method again.
- **Escape does nothing:** the **Pause Action** slot is empty.
- **The main menu is frozen after leaving a paused game:** `Time.timeScale = 1f;` is missing from `GoToMainMenu`.
- **Nothing is highlighted with the arrow keys:** a **First … Button** slot is empty.
- **NullReferenceException in MainMenu when you click Play:** you started from a scene without a GameManager or Scene Loader. Start from `S06_Menus`.
- **"GetVolume does not exist":** you changed VolumeSlider but not the AudioManager (part 1).

Try it

1. You can still attack with J while paused. Make SimpleAttack ignore input while paused: add `if (Time.timeScale == 0) return;` at the top of its `Update`.
2. Add a **Credits** panel to the main menu with a Back button.
3. Show the final score on the Game Over panel. (Hint: read `GameManager.Instance.Score` when it opens.)